Cálculo de Programas
Trabalho Prático
MiEI+LCC — 2020/21

Departamento de Informática
Universidade do Minho

Junho de 2021

Grupo nr.	50
a93300	Bohdan Malanka
a93278	Diogo da Silva Rebelo
a93316	Henrique José Fernandes Alvelos

1 Preâmbulo

Cálculo de Programas tem como objectivo principal ensinar a programação de computadores como uma disciplina científica. Para isso parte-se de um repertório de *combinadores* que formam uma álgebra da programação (conjunto de leis universais e seus corolários) e usam-se esses combinadores para construir programas *composicionalmente*, isto é, agregando programas já existentes.

Na sequência pedagógica dos planos de estudo dos dois cursos que têm esta disciplina, opta-se pela aplicação deste método à programação em Haskell (sem prejuízo da sua aplicação a outras linguagens funcionais). Assim, o presente trabalho prático coloca os alunos perante problemas concretos que deverão ser implementados em Haskell. Há ainda um outro objectivo: o de ensinar a documentar programas, a validá-los e a produzir textos técnico-científicos de qualidade.

2 Documentação

Para cumprir de forma integrada os objectivos enunciados acima vamos recorrer a uma técnica de programação dita "literária" [**?**], cujo princípio base é o seguinte:

> *Um programa e a sua documentação devem coincidir.*

Por outras palavras, o código fonte e a documentação de um programa deverão estar no mesmo ficheiro.

O ficheiro `cp2021t.pdf` que está a ler é já um exemplo de programação literária: foi gerado a partir do texto fonte `cp2021t.lhs`[1] que encontrará no material pedagógico desta disciplina descompactando o ficheiro `cp2021t.zip` e executando:

```
$ lhs2TeX cp2021t.lhs > cp2021t.tex
$ pdflatex cp2021t
```

em que `lhs2tex` é um pre-processador que faz "pretty printing" de código Haskell em LaTeX e que deve desde já instalar executando

```
$ cabal install lhs2tex --lib
```

Por outro lado, o mesmo ficheiro `cp2021t.lhs` é executável e contém o "kit" básico, escrito em Haskell, para realizar o trabalho. Basta executar

```
$ ghci cp2021t.lhs
```

[1] O suffixo 'lhs' quer dizer *literate Haskell*.

Abra o ficheiro `cp2021t.lhs` no seu editor de texto preferido e verifique que assim é: todo o texto que se encontra dentro do ambiente

```
\begin{code}
...
\end{code}
```

é seleccionado pelo GHCi para ser executado.

3 Como realizar o trabalho

Este trabalho teórico-prático deve ser realizado por grupos de 3 (ou 4) alunos. Os detalhes da avaliação (datas para submissão do relatório e sua defesa oral) são os que forem publicados na página da disciplina na *internet*.

Recomenda-se uma abordagem participativa dos membros do grupo de trabalho por forma a poderem responder às questões que serão colocadas na *defesa oral* do relatório.

Em que consiste, então, o *relatório* a que se refere o parágrafo anterior? É a edição do texto que está a ser lido, preenchendo o anexo D com as respostas. O relatório deverá conter ainda a identificação dos membros do grupo de trabalho, no local respectivo da folha de rosto.

Para gerar o PDF integral do relatório deve-se ainda correr os comando seguintes, que actualizam a bibliografia (com BibTEX) e o índice remissivo (com `makeindex`),

```
$ bibtex cp2021t.aux
$ makeindex cp2021t.idx
```

e recompilar o texto como acima se indicou. Dever-se-á ainda instalar o utilitário QuickCheck, que ajuda a validar programas em Haskell e a biblioteca Gloss para geração de gráficos 2D:

```
$ cabal install QuickCheck gloss --lib
```

Para testar uma propriedade QuickCheck *prop*, basta invocá-la com o comando:

```
> quickCheck prop
+++ OK, passed 100 tests.
```

Pode-se ainda controlar o número de casos de teste e sua complexidade, como o seguinte exemplo mostra:

```
> quickCheckWith stdArgs { maxSuccess = 200, maxSize = 10 } prop
+++ OK, passed 200 tests.
```

Qualquer programador tem, na vida real, de ler e analisar (muito!) código escrito por outros. No anexo C disponibiliza-se algum código Haskell relativo aos problemas que se seguem. Esse anexo deverá ser consultado e analisado à medida que isso for necessário.

3.1 Stack

O Stack é um programa útil para criar, gerir e manter projetos em Haskell. Um projeto criado com o Stack possui uma estrutura de pastas muito específica:

- Os módulos auxiliares encontram-se na pasta *src*.
- O módulos principal encontra-se na pasta *app*.
- A lista de depêndencias externas encontra-se no ficheiro *package.yaml*.

Pode aceder ao GHCi utilizando o comando:

```
stack ghci
```

Garanta que se encontra na pasta mais externa **do projeto**. A primeira vez que correr este comando as depêndencias externas serão instaladas automaticamente.

Para gerar o PDF, garanta que se encontra na diretoria *app*.

Problema 1

Os *tipos de dados algébricos* estudados ao longo desta disciplina oferecem uma grande capacidade expressiva ao programador. Graças à sua flexibilidade, torna-se trivial implementar DSLs e até mesmo linguagens de programação.

Paralelamente, um tópico bastante estudado no âmbito de Deep Learning é a derivação automática de expressões matemáticas, por exemplo, de derivadas. Duas técnicas que podem ser utilizadas para o cálculo de derivadas são:

- *Symbolic differentiation*
- *Automatic differentiation*

Symbolic differentiation consiste na aplicação sucessiva de transformações (leia-se: funções) que sejam congruentes com as regras de derivação. O resultado final será a expressão da derivada.

O leitor atento poderá notar um problema desta técnica: a expressão inicial pode crescer de forma descontrolada, levando a um cálculo pouco eficiente. *Automatic differentiation* tenta resolver este problema, calculando **o valor** da derivada da expressão em todos os passos. Para tal, é necessário calcular o valor da expressão **e** o valor da sua derivada.

Vamos de seguida definir uma linguagem de expressões matemáticas simples e implementar as duas técnicas de derivação automática. Para isso, seja dado o seguinte tipo de dados,

$$
\begin{array}{l}
\textbf{data}\ ExpAr\ a = X \\
\quad \mid N\ a \\
\quad \mid Bin\ BinOp\ (ExpAr\ a)\ (ExpAr\ a) \\
\quad \mid Un\ UnOp\ (ExpAr\ a) \\
\quad \textbf{deriving}\ (Eq, Show)
\end{array}
$$

onde $BinOp$ e $UnOp$ representam operações binárias e unárias, respectivamente:

$$
\begin{array}{l}
\textbf{data}\ BinOp = Sum \\
\quad \mid Product \\
\quad \textbf{deriving}\ (Eq, Show) \\
\textbf{data}\ UnOp = Negate \\
\quad \mid E \\
\quad \textbf{deriving}\ (Eq, Show)
\end{array}
$$

O construtor E simboliza o exponencial de base e.

Assim, cada expressão pode ser uma variável, um número, uma operação binária aplicada às devidas expressões, ou uma operação unária aplicada a uma expressão. Por exemplo,

$$Bin\ Sum\ X\ (N\ 10)$$

designa $x + 10$ na notação matemática habitual.

1. A definição das funções $inExpAr$ e $baseExpAr$ para este tipo é a seguinte:

$$
\begin{array}{l}
inExpAr = [\underline{X}, num_ops]\ \textbf{where} \\
\quad num_ops = [N, ops] \\
\quad ops = [bin, \widehat{Un}] \\
\quad bin\ (op, (a, b)) = Bin\ op\ a\ b \\
baseExpAr\ f\ g\ h\ j\ k\ l\ z = f + (g + (h \times (j \times k) + l \times z))
\end{array}
$$

Defina as funções $outExpAr$ e $recExpAr$, e teste as propriedades que se seguem.

Propriedade [QuickCheck] 1 *$inExpAr$ e $outExpAr$ são testemunhas de um isomorfismo, isto é, $inExpAr \cdot outExpAr = id$ e $outExpAr \cdot idExpAr = id$:*

$$
\begin{array}{l}
prop_in_out_idExpAr :: (Eq\ a) \Rightarrow ExpAr\ a \rightarrow Bool \\
prop_in_out_idExpAr = inExpAr \cdot outExpAr \equiv id \\
prop_out_in_idExpAr :: (Eq\ a) \Rightarrow OutExpAr\ a \rightarrow Bool \\
prop_out_in_idExpAr = outExpAr \cdot inExpAr \equiv id
\end{array}
$$

2. Dada uma expressão aritmética e um escalar para substituir o X, a função

$$eval_exp :: Floating\ a \Rightarrow a \rightarrow (ExpAr\ a) \rightarrow a$$

calcula o resultado da expressão. Na página 12 esta função está expressa como um catamorfismo. Defina o respectivo gene e, de seguida, teste as propriedades:

Propriedade [QuickCheck] 2 *A função eval_exp respeita os elementos neutros das operações.*

$$\begin{array}{l}
prop_sum_idr :: (Floating\ a, Real\ a) \Rightarrow a \rightarrow ExpAr\ a \rightarrow Bool \\
prop_sum_idr\ a\ exp = eval_exp\ a\ exp \stackrel{?}{=} sum_idr\ \textbf{where} \\
\quad sum_idr = eval_exp\ a\ (Bin\ Sum\ exp\ (N\ 0)) \\
prop_sum_idl :: (Floating\ a, Real\ a) \Rightarrow a \rightarrow ExpAr\ a \rightarrow Bool \\
prop_sum_idl\ a\ exp = eval_exp\ a\ exp \stackrel{?}{=} sum_idl\ \textbf{where} \\
\quad sum_idl = eval_exp\ a\ (Bin\ Sum\ (N\ 0)\ exp) \\
prop_product_idr :: (Floating\ a, Real\ a) \Rightarrow a \rightarrow ExpAr\ a \rightarrow Bool \\
prop_product_idr\ a\ exp = eval_exp\ a\ exp \stackrel{?}{=} prod_idr\ \textbf{where} \\
\quad prod_idr = eval_exp\ a\ (Bin\ Product\ exp\ (N\ 1)) \\
prop_product_idl :: (Floating\ a, Real\ a) \Rightarrow a \rightarrow ExpAr\ a \rightarrow Bool \\
prop_product_idl\ a\ exp = eval_exp\ a\ exp \stackrel{?}{=} prod_idl\ \textbf{where} \\
\quad prod_idl = eval_exp\ a\ (Bin\ Product\ (N\ 1)\ exp) \\
prop_e_id :: (Floating\ a, Real\ a) \Rightarrow a \rightarrow Bool \\
prop_e_id\ a = eval_exp\ a\ (Un\ E\ (N\ 1)) \equiv expd\ 1 \\
prop_negate_id :: (Floating\ a, Real\ a) \Rightarrow a \rightarrow Bool \\
prop_negate_id\ a = eval_exp\ a\ (Un\ Negate\ (N\ 0)) \equiv 0
\end{array}$$

Propriedade [QuickCheck] 3 *Negar duas vezes uma expressão tem o mesmo valor que não fazer nada.*

$$\begin{array}{l}
prop_double_negate :: (Floating\ a, Real\ a) \Rightarrow a \rightarrow ExpAr\ a \rightarrow Bool \\
prop_double_negate\ a\ exp = eval_exp\ a\ exp \stackrel{?}{=} eval_exp\ a\ (Un\ Negate\ (Un\ Negate\ exp))
\end{array}$$

3. É possível otimizar o cálculo do valor de uma expressão aritmética tirando proveito dos elementos absorventes de cada operação. Implemente os genes da função

$$optmize_eval :: (Floating\ a, Eq\ a) \Rightarrow a \rightarrow (ExpAr\ a) \rightarrow a$$

que se encontra na página 12 expressa como um hilomorfismo[2] e teste as propriedades:

Propriedade [QuickCheck] 4 *A função optimize_eval respeita a semântica da função eval.*

$$\begin{array}{l}
prop_optimize_respects_semantics :: (Floating\ a, Real\ a) \Rightarrow a \rightarrow ExpAr\ a \rightarrow Bool \\
prop_optimize_respects_semantics\ a\ exp = eval_exp\ a\ exp \stackrel{?}{=} optmize_eval\ a\ exp
\end{array}$$

4. Para calcular a derivada de uma expressão, é necessário aplicar transformações à expressão original que respeitem as regras das derivadas:[3]

   - Regra da soma:

$$\frac{d}{dx}(f(x) + g(x)) = \frac{d}{dx}(f(x)) + \frac{d}{dx}(g(x))$$

[2] Qual é a vantagem de implementar a função *optimize_eval* utilizando um hilomorfismo em vez de utilizar um catamorfismo com um gene "inteligente"?

[3] Apesar da adição e multiplicação gozarem da propriedade comutativa, há que ter em atenção a ordem das operações por causa dos testes.

- Regra do produto:

$$\frac{d}{dx}(f(x)g(x)) = f(x) \cdot \frac{d}{dx}(g(x)) + \frac{d}{dx}(f(x)) \cdot g(x)$$

Defina o gene do catamorfismo que ocorre na função

$$sd :: Floating\ a \Rightarrow ExpAr\ a \rightarrow ExpAr\ a$$

que, dada uma expressão aritmética, calcula a sua derivada. Testes a fazer, de seguida:

Propriedade [QuickCheck] 5 *A função sd respeita as regras de derivação.*

$$\begin{array}{l}
prop_const_rule :: (Real\ a, Floating\ a) \Rightarrow a \rightarrow Bool \\
prop_const_rule\ a = sd\ (N\ a) \equiv N\ 0 \\
prop_var_rule :: Bool \\
prop_var_rule = sd\ X \equiv N\ 1 \\
prop_sum_rule :: (Real\ a, Floating\ a) \Rightarrow ExpAr\ a \rightarrow ExpAr\ a \rightarrow Bool \\
prop_sum_rule\ exp1\ exp2 = sd\ (Bin\ Sum\ exp1\ exp2) \equiv sum_rule\ \mathbf{where} \\
\quad sum_rule = Bin\ Sum\ (sd\ exp1)\ (sd\ exp2) \\
prop_product_rule :: (Real\ a, Floating\ a) \Rightarrow ExpAr\ a \rightarrow ExpAr\ a \rightarrow Bool \\
prop_product_rule\ exp1\ exp2 = sd\ (Bin\ Product\ exp1\ exp2) \equiv prod_rule\ \mathbf{where} \\
\quad prod_rule = Bin\ Sum\ (Bin\ Product\ exp1\ (sd\ exp2))\ (Bin\ Product\ (sd\ exp1)\ exp2) \\
prop_e_rule :: (Real\ a, Floating\ a) \Rightarrow ExpAr\ a \rightarrow Bool \\
prop_e_rule\ exp = sd\ (Un\ E\ exp) \equiv Bin\ Product\ (Un\ E\ exp)\ (sd\ exp) \\
prop_negate_rule :: (Real\ a, Floating\ a) \Rightarrow ExpAr\ a \rightarrow Bool \\
prop_negate_rule\ exp = sd\ (Un\ Negate\ exp) \equiv Un\ Negate\ (sd\ exp)
\end{array}$$

5. Como foi visto, *Symbolic differentiation* não é a técnica mais eficaz para o cálculo do valor da derivada de uma expressão. *Automatic differentiation* resolve este problema cálculando o valor da derivada em vez de manipular a expressão original.

   Defina o gene do catamorfismo que ocorre na função

$$ad :: Floating\ a \Rightarrow a \rightarrow ExpAr\ a \rightarrow a$$

   que, dada uma expressão aritmética e um ponto, calcula o valor da sua derivada nesse ponto, sem transformar manipular a expressão original. Testes a fazer, de seguida:

   Propriedade [QuickCheck] 6 *Calcular o valor da derivada num ponto r via ad é equivalente a calcular a derivada da expressão e avalia-la no ponto r.*

$$\begin{array}{l}
prop_congruent :: (Floating\ a, Real\ a) \Rightarrow a \rightarrow ExpAr\ a \rightarrow Bool \\
prop_congruent\ a\ exp = ad\ a\ exp \stackrel{?}{=} eval_exp\ a\ (sd\ exp)
\end{array}$$

Problema 2

Nesta disciplina estudou-se como fazer programação dinâmica por cálculo, recorrendo à lei de recursividade mútua.[4]

Para o caso de funções sobre os números naturais ($\mathbb{N}_0$, com functor $\mathsf{F}\ X = 1 + X$) é fácil derivar-se da lei que foi estudada uma *regra de algibeira* que se pode ensinar a programadores que não tenham estudado Cálculo de Programas. Apresenta-se de seguida essa regra, tomando como exemplo o cálculo do ciclo-for que implementa a função de Fibonacci, recordar o sistema

$$\begin{array}{l}
fib\ 0 = 1 \\
fib\ (n+1) = f\ n
\end{array}$$

[4] Lei (3.94) em [?], página 98.

$$
\begin{aligned}
& f\ 0 = 1 \\
& f\ (n+1) = \mathit{fib}\ n + f\ n
\end{aligned}
$$

Obter-se-á de imediato

$$
\begin{aligned}
& \mathit{fib}' = \pi_1 \cdot \mathsf{for}\ \mathit{loop}\ \mathit{init}\ \mathbf{where} \\
& \quad \mathit{loop}\ (\mathit{fib}, f) = (f, \mathit{fib} + f) \\
& \quad \mathit{init} = (1, 1)
\end{aligned}
$$

usando as regras seguintes:

- O corpo do ciclo *loop* terá tantos argumentos quanto o número de funções mutuamente recursivas.
- Para as variáveis escolhem-se os próprios nomes das funções, pela ordem que se achar conveniente.[5]
- Para os resultados vão-se buscar as expressões respectivas, retirando a variável n.
- Em *init* coleccionam-se os resultados dos casos de base das funções, pela mesma ordem.

Mais um exemplo, envolvendo polinómios do segundo grau $ax^2 + bx + c$ em $\mathbb{N}_0$. Seguindo o método estudado nas aulas[6], de $f\ x = ax^2 + bx + c$ derivam-se duas funções mutuamente recursivas:

$$
\begin{aligned}
& f\ 0 = c \\
& f\ (n+1) = f\ n + k\ n \\
& k\ 0 = a + b \\
& k\ (n+1) = k\ n + 2\ a
\end{aligned}
$$

Seguindo a regra acima, calcula-se de imediato a seguinte implementação, em Haskell:

$$
\begin{aligned}
& f'\ a\ b\ c = \pi_1 \cdot \mathsf{for}\ \mathit{loop}\ \mathit{init}\ \mathbf{where} \\
& \quad \mathit{loop}\ (f, k) = (f + k, k + 2 * a) \\
& \quad \mathit{init} = (c, a + b)
\end{aligned}
$$

O que se pede então, nesta pergunta? Dada a fórmula que dá o n-ésimo número de Catalan,

$$
C_n = \frac{(2n)!}{(n+1)!(n!)} \tag{1}
$$

derivar uma implementação de C_n que não calcule factoriais nenhuns. Isto é, derivar um ciclo-for

$$
\mathit{cat} = \cdots \cdot \mathsf{for}\ \mathit{loop}\ \mathit{init}\ \mathbf{where}\ \cdots
$$

que implemente esta função.

Propriedade [QuickCheck] 7 *A função proposta coincidem com a definição dada:*

$$
\mathit{prop_cat} = (\geqslant 0) \Rightarrow (\mathit{catdef} \equiv \mathit{cat})
$$

Sugestão: Começar por estudar muito bem o processo de cálculo dado no anexo B para o problema (semelhante) da função exponencial.

Problema 3

As curvas de Bézier, designação dada em honra ao engenheiro Pierre Bézier, são curvas ubíquas na área de computação gráfica, animação e modelação. Uma curva de Bézier é uma curva paramétrica, definida por um conjunto $\{P_0, ..., P_N\}$ de pontos de controlo, onde N é a ordem da curva.

O algoritmo de *De Casteljau* é um método recursivo capaz de calcular curvas de Bézier num ponto. Apesar de ser mais lento do que outras abordagens, este algoritmo é numericamente mais estável, trocando velocidade por correção.

[5] Podem obviamente usar-se outros símbolos, mas numa primeira leitura dá jeito usarem-se tais nomes.
[6] Secção 3.17 de [**?**] e tópico Recursividade mútua nos vídeos das aulas teóricas.

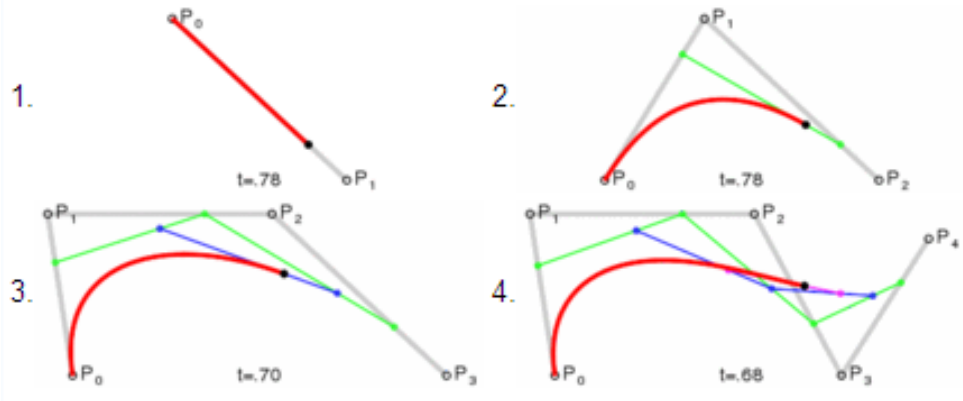

Figura 1: Exemplos de curvas de Bézier retirados da Wikipedia.

De forma sucinta, o valor de uma curva de Bézier de um só ponto $\{P_0\}$ (ordem 0) é o próprio ponto P_0. O valor de uma curva de Bézier de ordem N é calculado através da interpolação linear da curva de Bézier dos primeiros $N-1$ pontos e da curva de Bézier dos últimos $N-1$ pontos.

A interpolação linear entre 2 números, no intervalo $[0,1]$, é dada pela seguinte função:

$$
\begin{array}{l}
\mathit{linear1d} :: \mathbb{Q} \to \mathbb{Q} \to \mathit{OverTime}\ \mathbb{Q} \\
\mathit{linear1d}\ a\ b = \mathit{formula}\ a\ b\ \textbf{where} \\
\quad \mathit{formula} :: \mathbb{Q} \to \mathbb{Q} \to \mathit{Float} \to \mathbb{Q} \\
\quad \mathit{formula}\ x\ y\ t = ((1.0 :: \mathbb{Q}) - (\mathit{to}_{\mathbb{Q}}\ t)) * x + (\mathit{to}_{\mathbb{Q}}\ t) * y
\end{array}
$$

A interpolação linear entre 2 pontos de dimensão N é calculada através da interpolação linear de cada dimensão.

O tipo de dados NPoint representa um ponto com N dimensões.

$$\textbf{type}\ \mathit{NPoint} = [\mathbb{Q}]$$

Por exemplo, um ponto de 2 dimensões e um ponto de 3 dimensões podem ser representados, respetivamente, por:

$$
\begin{array}{l}
\mathit{p2d} = [1.2, 3.4] \\
\mathit{p3d} = [0.2, 10.3, 2.4]
\end{array}
$$

O tipo de dados $\mathit{OverTime}\ a$ representa um termo do tipo a num dado instante (dado por um Float).

$$\textbf{type}\ \mathit{OverTime}\ a = \mathit{Float} \to a$$

O anexo C tem definida a função

$$\mathit{calcLine} :: \mathit{NPoint} \to (\mathit{NPoint} \to \mathit{OverTime}\ \mathit{NPoint})$$

que calcula a interpolação linear entre 2 pontos, e a função

$$\mathit{deCasteljau} :: [\mathit{NPoint}] \to \mathit{OverTime}\ \mathit{NPoint}$$

que implementa o algoritmo respectivo.

1. Implemente $\mathit{calcLine}$ como um catamorfismo de listas, testando a sua definição com a propriedade:

   Propriedade [QuickCheck] 8 *Definição alternativa.*

   $$
   \begin{array}{l}
   \mathit{prop_calcLine_def} :: \mathit{NPoint} \to \mathit{NPoint} \to \mathit{Float} \to \mathit{Bool} \\
   \mathit{prop_calcLine_def}\ p\ q\ d = \mathit{calcLine}\ p\ q\ d \equiv \mathit{zipWithM}\ \mathit{linear1d}\ p\ q\ d
   \end{array}
   $$

2. Implemente a função $\mathit{deCasteljau}$ como um hilomorfismo, testando agora a propriedade:

Propriedade [QuickCheck] 9 *Curvas de Bézier são simétricas.*

$$
\begin{array}{l}
prop_bezier_sym :: [[\mathbb{Q}]] \to Gen\ Bool \\
prop_bezier_sym\ l = all\ (<\Delta) \cdot calc_difs \cdot bezs\ \langle \$ \rangle\ elements\ ps\ \mathbf{where} \\
\quad calc_difs = (\lambda(x,y) \to zipWith\ (\lambda w\ v \to \mathbf{if}\ w \geqslant v\ \mathbf{then}\ w - v\ \mathbf{else}\ v - w)\ x\ y) \\
\quad bezs\ t = (deCasteljau\ l\ t, deCasteljau\ (reverse\ l)\ (from_{\mathbb{Q}}\ (1 - (to_{\mathbb{Q}}\ t)))) \\
\quad \Delta = 1\mathrm{e}{-2}
\end{array}
$$

3. Corra a função $runBezier$ e aprecie o seu trabalho[7] clicando na janela que é aberta (que contém, a verde, um ponto inicila) com o botão esquerdo do rato para adicionar mais pontos. A tecla $Delete$ apaga o ponto mais recente.

Problema 4

Seja dada a fórmula que calcula a média de uma lista não vazia x,

$$avg\ x = \frac{1}{k} \sum_{i=1}^{k} x_i \tag{2}$$

onde $k = length\ x$. Isto é, para sabermos a média de uma lista precisamos de dois catamorfismos: o que faz o somatório e o que calcula o comprimento a lista. Contudo, é facil de ver que

$$
\begin{array}{l}
avg\ [a] = a \\
avg(a : x) = \frac{1}{k+1}(a + \sum_{i=1}^{k} x_i) = \frac{a + k(avg\ x)}{k+1} \text{ para } k = length\ x
\end{array}
$$

Logo avg está em recursividade mútua com $length$ e o par de funções pode ser expresso por um único catamorfismo, significando que a lista apenas é percorrida uma vez.

1. Recorra à lei de recursividade mútua para derivar a função $avg_aux = (\!|[b, q]|\!)$ tal que $avg_aux = \langle avg, length \rangle$ em listas não vazias.

2. Generalize o raciocínio anterior para o cálculo da média de todos os elementos de uma LTree recorrendo a uma única travessia da árvore (i.e. catamorfismo).

Verifique as suas funções testando a propriedade seguinte:

Propriedade [QuickCheck] 10 *A média de uma lista não vazia e de uma LTree com os mesmos elementos coincide, a menos de um erro de 0.1 milésimas:*

$$
\begin{array}{l}
prop_avg :: [Double] \to Property \\
prop_avg = nonempty \Rightarrow diff \leqslant \underline{0.000001}\ \mathbf{where} \\
\quad diff\ l = avg\ l - (avgLTree \cdot genLTree)\ l \\
\quad genLTree = [\!(lsplit)\!] \\
\quad nonempty = (>[\,])
\end{array}
$$

Problema 5

(**NB**: Esta questão é **opcional** e funciona como **valorização** apenas para os alunos que desejarem fazê-la.)

Existem muitas linguagens funcionais para além do Haskell, que é a linguagem usada neste trabalho prático. Uma delas é o F# da Microsoft. Na directoria `fsharp` encontram-se os módulos Cp, Nat e LTree codificados em F#. O que se pede é a biblioteca BTree escrita na mesma linguagem.

Modo de execução: o código que tiverem produzido nesta pergunta deve ser colocado entre o `\begin{verbatim}` e o `\end{verbatim}` da correspondente parte do anexo D. Para além disso, os grupos podem demonstrar o código na oral.

[7] A representação em Gloss é uma adaptação de um projeto de Harold Cooper.

Anexos

A Como exprimir cálculos e diagramas em LaTeX/lhs2tex

Como primeiro exemplo, estudar o texto fonte deste trabalho para obter o efeito:[8]

$$
\begin{array}{ll}
 & id = \langle f, g \rangle \\
\equiv & \qquad \{ \text{ universal property } \} \\
 & \left\{ \begin{array}{l} \pi_1 \cdot id = f \\ \pi_2 \cdot id = g \end{array} \right. \\
\equiv & \qquad \{ \text{ identity } \} \\
 & \left\{ \begin{array}{l} \pi_1 = f \\ \pi_2 = g \end{array} \right. \\
\Box &
\end{array}
$$

Os diagramas podem ser produzidos recorrendo à *package* LaTeX xymatrix, por exemplo:

$$
\begin{tikzcd}
\mathbb{N}_0 \arrow[d, "(\!|g|\!)"'] & 1 + \mathbb{N}_0 \arrow[l, "\mathsf{in}"'] \arrow[d, "id + (\!|g|\!)"] \\
B & 1 + B \arrow[l, "g"]
\end{tikzcd}
$$

B Programação dinâmica por recursividade múltipla

Neste anexo dão-se os detalhes da resolução do Exercício 3.30 dos apontamentos da disciplina[9], onde se pretende implementar um ciclo que implemente o cálculo da aproximação até $i = n$ da função exponencial $exp\ x = e^x$, via série de Taylor:

$$
exp\ x \quad = \quad \sum_{i=0}^{\infty} \frac{x^i}{i!} \tag{3}
$$

Seja $e\ x\ n = \sum_{i=0}^{n} \frac{x^i}{i!}$ a função que dá essa aproximação. É fácil de ver que $e\ x\ 0 = 1$ e que $e\ x\ (n+1) = e\ x\ n + \frac{x^{n+1}}{(n+1)!}$. Se definirmos $h\ x\ n = \frac{x^{n+1}}{(n+1)!}$ teremos $e\ x$ e $h\ x$ em recursividade mútua. Se repetirmos o processo para $h\ x\ n$ etc obteremos no total três funções nessa mesma situação:

$$
\begin{array}{l}
e\ x\ 0 = 1 \\
e\ x\ (n+1) = h\ x\ n + e\ x\ n \\
h\ x\ 0 = x \\
h\ x\ (n+1) = x\ /\ (s\ n) * h\ x\ n \\
s\ 0 = 2 \\
s\ (n+1) = 1 + s\ n
\end{array}
$$

Segundo a *regra de algibeira* descrita na página 3.1 deste enunciado, ter-se-á, de imediato:

$$
\begin{array}{l}
e'\ x = prj \cdot \mathsf{for}\ loop\ init\ \mathbf{where} \\
\quad init = (1, x, 2) \\
\quad loop\ (e, h, s) = (h + e, x\ /\ s * h, 1 + s) \\
\quad prj\ (e, h, s) = e
\end{array}
$$

[8] Exemplos tirados de [?].
[9] Cf. [?], página 102.

C Código fornecido

Problema 1

$$
\begin{aligned}
&expd :: Floating\ a \Rightarrow a \to a \\
&expd = Prelude.exp \\
&\mathbf{type}\ OutExpAr\ a = () + (a + ((BinOp, (ExpAr\ a, ExpAr\ a)) + (UnOp, ExpAr\ a)))
\end{aligned}
$$

Problema 2

Definição da série de Catalan usando factoriais (1):

$$catdef\ n = (2 * n)! \div ((n+1)! * n!)$$

Oráculo para inspecção dos primeiros 26 números de Catalan[10]:

$$
\begin{aligned}
&oracle = [\\
&\quad 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, \\
&\quad 35357670, 129644790, 477638700, 1767263190, 6564120420, 24466267020, \\
&\quad 91482563640, 343059613650, 1289904147324, 4861946401452 \\
&\quad]
\end{aligned}
$$

Problema 3

Algoritmo:

$$
\begin{aligned}
&deCasteljau :: [NPoint] \to OverTime\ NPoint \\
&deCasteljau\ [\,] = nil \\
&deCasteljau\ [p] = \underline{p} \\
&deCasteljau\ l = \lambda pt \to (calcLine\ (p\ pt)\ (q\ pt))\ pt\ \mathbf{where} \\
&\quad p = deCasteljau\ (init\ l) \\
&\quad q = deCasteljau\ (tail\ l)
\end{aligned}
$$

Função auxiliar:

$$
\begin{aligned}
&calcLine :: NPoint \to (NPoint \to OverTime\ NPoint) \\
&calcLine\ [\,] = \underline{nil} \\
&calcLine\ (p : x) = \overline{g}\ p\ (calcLine\ x)\ \mathbf{where} \\
&\quad g :: (\mathbb{Q}, NPoint \to OverTime\ NPoint) \to (NPoint \to OverTime\ NPoint) \\
&\quad g\ (d, f)\ l = \mathbf{case}\ l\ \mathbf{of} \\
&\qquad [\,] \to nil \\
&\qquad (x : xs) \to \lambda z \to concat\ \$\ (sequenceA\ [singl \cdot linear1d\ d\ x, f\ xs])\ z
\end{aligned}
$$

2D:

$$
\begin{aligned}
&bezier2d :: [NPoint] \to OverTime\ (Float, Float) \\
&bezier2d\ [\,] = \underline{(0, 0)} \\
&bezier2d\ l = \lambda z \to (from_{\mathbb{Q}} \times from_{\mathbb{Q}}) \cdot (\lambda [x, y] \to (x, y))\ \$\ ((deCasteljau\ l)\ z)
\end{aligned}
$$

Modelo:

$$
\begin{aligned}
&\mathbf{data}\ World = World\ \{points :: [NPoint] \\
&\quad , time :: Float \\
&\quad \} \\
&initW :: World \\
&initW = World\ [\,]\ 0
\end{aligned}
$$

[10] Fonte: Wikipedia.

```
tick :: Float → World → World
tick dt world = world {time = (time world) + dt}

actions :: Event → World → World
actions (EventKey (MouseButton LeftButton) Down _ p) world =
  world {points = (points world) ++ [(λ(x, y) → map toQ [x, y]) p]}
actions (EventKey (SpecialKey KeyDelete) Down _ _) world =
  world {points = cond (≡ []) id init (points world)}
actions _ world = world

scaleTime :: World → Float
scaleTime w = (1 + cos (time w)) / 2

bezier2dAtTime :: World → (Float, Float)
bezier2dAtTime w = (bezier2dAt w) (scaleTime w)

bezier2dAt :: World → OverTime (Float, Float)
bezier2dAt w = bezier2d (points w)

thicCirc :: Picture
thicCirc = ThickCircle 4 10

ps :: [Float]
ps = map fromQ ps' where
  ps' :: [Q]
  ps' = [0, 0.01 .. 1]   -- interval
```

Gloss:

```
picture :: World → Picture
picture world = Pictures
  [animateBezier (scaleTime world) (points world)
  , Color white · Line · map (bezier2dAt world) $ ps
  , Color blue · Pictures $ [Translate (fromQ x) (fromQ y) thicCirc | [x, y] ← points world]
  , Color green $ Translate cx cy thicCirc
  ] where
  (cx, cy) = bezier2dAtTime world
```

Animação:

```
animateBezier :: Float → [NPoint] → Picture
animateBezier _ [] = Blank
animateBezier _ [_] = Blank
animateBezier t l = Pictures
  [animateBezier t (init l)
  , animateBezier t (tail l)
  , Color red · Line $ [a, b]
  , Color orange $ Translate ax ay thicCirc
  , Color orange $ Translate bx by thicCirc
  ] where
  a@(ax, ay) = bezier2d (init l) t
  b@(bx, by) = bezier2d (tail l) t
```

Propriedades e *main*:

```
runBezier :: IO ()
runBezier = play (InWindow "Bézier" (600, 600) (0, 0))
  black 50 initW picture actions tick

runBezierSym :: IO ()
runBezierSym = quickCheckWith (stdArgs {maxSize = 20, maxSuccess = 200}) prop_bezier_sym
```

Compilação e execução dentro do interpretador:[11]

```
main = runBezier

run = do {system "ghc cp2021t"; system "./cp2021t"}
```

[11] Pode ser útil em testes envolvendo Gloss. Nesse caso, o teste em causa deve fazer parte de uma função *main*.

QuickCheck

Código para geração de testes:

```
instance Arbitrary UnOp where
  arbitrary = elements [Negate, E]
instance Arbitrary BinOp where
  arbitrary = elements [Sum, Product]
instance (Arbitrary a) ⇒ Arbitrary (ExpAr a) where
  arbitrary = do
    binop ← arbitrary
    unop ← arbitrary
    exp1 ← arbitrary
    exp2 ← arbitrary
    a    ← arbitrary
    frequency · map (id × pure) $ [(20, X), (15, N a), (35, Bin binop exp1 exp2), (30, Un unop exp1)]
infixr 5 ≟
(≟) :: Real a ⇒ a → a → Bool
(≟) x y = (to_ℚ x) ≡ (to_ℚ y)
```

Outras funções auxiliares

Lógicas:

```
infixr 0 ⇒
(⇒) :: (Testable prop) ⇒ (a → Bool) → (a → prop) → a → Property
p ⇒ f = λa → p a ⇒ f a
infixr 0 ⇔
(⇔) :: (a → Bool) → (a → Bool) → a → Property
p ⇔ f = λa → (p a ⇒ property (f a)) .&&. (f a ⇒ property (p a))
infixr 4 ≡
(≡) :: Eq b ⇒ (a → b) → (a → b) → (a → Bool)
f ≡ g = λa → f a ≡ g a
infixr 4 ⩽
(⩽) :: Ord b ⇒ (a → b) → (a → b) → (a → Bool)
f ⩽ g = λa → f a ⩽ g a
infixr 4 ∧
(∧) :: (a → Bool) → (a → Bool) → (a → Bool)
f ∧ g = λa → ((f a) ∧ (g a))
```

D Soluções dos alunos

Os alunos devem colocar neste anexo as suas soluções para os exercícios propostos, de acordo com o "layout"que se fornece. Não podem ser alterados os nomes ou tipos das funções dadas, mas pode ser adicionado texto, disgramas e/ou outras funções auxiliares que sejam necessárias.

Valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes.

Problema 1

São dadas:

```
cataExpAr g = g · recExpAr (cataExpAr g) · outExpAr
anaExpAr g = inExpAr · recExpAr (anaExpAr g) · g
hyloExpAr h g = cataExpAr h · anaExpAr g
```

$$
\begin{aligned}
&eval_exp :: Floating\ a \Rightarrow a \to (ExpAr\ a) \to a \\
&eval_exp\ a = cataExpAr\ (g_eval_exp\ a) \\
&optmize_eval :: (Floating\ a, Eq\ a) \Rightarrow a \to (ExpAr\ a) \to a \\
&optmize_eval\ a = hyloExpAr\ (gopt\ a)\ clean \\
&sd :: Floating\ a \Rightarrow ExpAr\ a \to ExpAr\ a \\
&sd = \pi_2 \cdot cataExpAr\ sd_gen \\
&ad :: Floating\ a \Rightarrow a \to ExpAr\ a \to a \\
&ad\ v = \pi_2 \cdot cataExpAr\ (ad_gen\ v)
\end{aligned}
$$

Definir:

$$
\begin{aligned}
& outExpAr \cdot inExpAr = id && (4) \\
\equiv \quad & \{ \text{ Definição de inExpAr } \} \\
& outExpAr \cdot [\underline{X}, [N, [bin, \widehat{Un}]]] = id && (5) \\
\equiv \quad & \{ \text{ Fusão-+, Universal-+ e Natural-+ } \} \\
& [outExpAr \cdot \underline{X}, outExpAr \cdot [N, [bin, \widehat{Un}]]] = i_1 \cdot i_2 && (6) \\
\equiv \quad & \{ \text{ Eq-+ } \} \\
& \left\{ \begin{array}{l} outExpAr \cdot \underline{X} = i_1 \\ outExpAr \cdot N = i_2 \cdot i_1 \\ outExpAr \cdot bin = i_2 \cdot i_2 \cdot i_1 \\ outExpAr \cdot \widehat{Un} = i_2 \cdot i_2 \cdot i_2 \end{array} \right. && (7) \\
\square && (8)
\end{aligned}
$$

$$
\begin{aligned}
&outExpAr\ X = i_1\ () \\
&outExpAr\ (N\ a) = i_2\ (i_1\ a) \\
&outExpAr\ (Bin\ op\ a\ b) = i_2\ (i_2\ (i_1\ (op, (a, b)))) \\
&outExpAr\ (Un\ op\ a) = i_2\ (i_2\ (i_2\ (op, a)))
\end{aligned}
$$

recExpr

$$
\begin{tikzcd}
ExprArA \arrow[rr, bend left, "outExprAr"] \arrow[d, "f"'] & \cong & 1 + (A + (BinOp \times (ExprArA \times ExprArA) + UnOp \times ExpArA)) \arrow[ll, bend left, "inExprAr"] \arrow[d, "id+(id+(id\times(f\times f)+id\times f))"] \\
a & & 1 + (A + (BinOp \times (a \times a) + UnOp \times a)) \arrow[ll]
\end{tikzcd}
$$

$$
\begin{aligned}
& recExpAr\ f = id + (id + (id \times (f \times f) + id \times f)) \\
\equiv \quad & \{ \text{ Aplicando a definição dada de baseExprAr } \} \\
& recExpAr\ f = baseExpAr\ id\ id\ id\ f\ f\ id\ f
\end{aligned}
$$

$$
recExpAr\ f = baseExpAr\ id\ id\ id\ f\ f\ id\ f
$$

—

g_eval_exp

Por forma a obtermos o valor final de uma expressão, podemos recorrer ao uso do catamorfismo de *ExprAr*, definindo o diagrama de *eval_exp* da seguinte maneira.

$$
\begin{tikzcd}
ExpArA \arrow[rr, bend left, "out"] \arrow[d, "eval_exp"'] & \cong & 1 + (A + (BinOp \times (ExpArA \times ExpArA)) + (UnOp \times ExpArA)) \arrow[ll, bend left, "\mathbf{in}"] \arrow[d, "id+(id+(id\times(eval_exp\times eval_exp)+id\times eval_exp))"] \\
a & & 1 + (a + ((BinOp \times (a \times a)) + UnOp \times a)) \arrow[ll, "g_eval_exp"]
\end{tikzcd}
$$

Como cada coproduto corresponde a um either, então através do diagrama podesmos chegar ao seguinte gene do catamorfismo eval_exp:

$$
\begin{array}{l}
g_eval_exp\ a = [g1\ \ a, g2]\ \mathbf{where} \\
\quad g1\ \ a = \underline{a} \\
\quad g2 = [g3, g4]\ \mathbf{where} \\
\qquad g3 = id \\
\qquad g4 = [g5, g6]\ \mathbf{where} \\
\qquad\quad g5\ (Sum, (a, b)) = a + b \\
\qquad\quad g5\ (Product, (a, b)) = a * b \\
\qquad\quad g6\ (Negate, a) = -a \\
\qquad\quad g6\ (E, a) = expd\ a
\end{array}
$$

—

optimize_eval

Reparamos que as resras matemáticas aqui só se aplicavam ao Product e o resto decidimos manter parecisso á função outExpAr, para que a estrutura de dados do anamorfismo se mantivesse. Assim, isto permitiu que o catamorfismo se mantivesse igual ao anterio.

$$
\begin{array}{l}
clean\ X = i_1\ () \\
clean\ (N\ a) = i_2\ (i_1\ a) \\
clean\ (Bin\ op\ a\ b) = \mathbf{if}\ op \equiv Product \wedge (a \equiv N\ 0 \vee b \equiv N\ 0)\ \mathbf{then}\ i_2\ (i_1\ 0)\ \mathbf{else}\ i_2\ (i_2\ (i_1\ (op, (a, b)))) \\
clean\ (Un\ op\ a) = i_2\ (i_2\ (i_2\ (op, a))) \\
\quad \text{--} \\
gopt\ a = g_eval_exp\ a
\end{array}
$$

—

$$
\begin{tikzcd}
ExpArA \arrow[rr, bend left, "out"] \arrow[d, "sd"'] & \cong & 1 + (A + (BinOp \times (ExpArA \times ExpArA)) + (UnOp \times ExpArA)) \arrow[ll, bend left, "\mathbf{in}"] \arrow[d, "id+(id+(id\times(sd\times sd)+id\times sd))"] \\
ExpAr\ A \times ExpAr\ A & & 1 + (A + ((BinOp \times ((ExpArA)^2 \times (ExpArA)^2)) + UnOp \times (ExpArA)^2)) \arrow[ll, "sd_gen"]
\end{tikzcd}
$$

Como cada coproduto corresponde a um either, então através do diagrama podesmos chegar ao seguinte gene do catamorfismo eval_exp, aplicando as regras de derivação da soma e do produto:

$$
\begin{array}{l}
sd_gen :: Floating\ a \Rightarrow \\
\quad () + (a + ((BinOp, ((ExpAr\ a, ExpAr\ a), (ExpAr\ a, ExpAr\ a))) + (UnOp, (ExpAr\ a, ExpAr\ a)))) \\
\quad\ \rightarrow (ExpAr\ a, ExpAr\ a) \\
sd_gen = [g1, g2]\ \mathbf{where} \\
\quad g1 = \underline{(X, N\ 1)} \\
\quad g2 = [g3, g4]\ \mathbf{where} \\
\quad\quad g3\ a = (N\ a, N\ 0) \\
\quad\quad g4 = [g5, g6]\ \mathbf{where} \\
\quad\quad\quad g5\ (Sum, ((ex1, ex2), (ex3, ex4))) = (Bin\ Sum\ ex1\ ex3, Bin\ Sum\ ex2\ ex4) \\
\quad\quad\quad g5\ (Product, ((ex1, ex2), (ex3, ex4))) = \\
\quad\quad\quad\quad (Bin\ Product\ ex1\ ex3, Bin\ Sum\ (Bin\ Product\ ex1\ ex4)\ (Bin\ Product\ ex2\ ex3)) \\
\quad\quad\quad g6\ (Negate, (ex1, ex2)) = (Un\ Negate\ ex1, Un\ Negate\ ex2) \\
\quad\quad\quad g6\ (E, (ex1, ex2)) = (Un\ E\ ex1, Bin\ Product\ (Un\ E\ ex1)\ ex2)
\end{array}
$$

—

ad_gen

$$
\begin{tikzcd}
ExpArA \arrow[rr, bend left, "out"] \arrow[d, "(\!|ad_gen|\!)=f"'] & \cong & 1 + (A + (BinOp \times (ExpArA \times ExpArA)) + (UnOp \times ExpArA)) \arrow[ll, bend left, "\mathbf{in}"] \arrow[d, "id+(id+(id\times(f\times f)+id\times f))"] \\
A \times A & & 1 + (A + ((BinOp \times ((ExpArA)^2 \times (ExpArA)^2)) + UnOp \times (ExpArA)^2)) \arrow[ll, "ad_gen"]
\end{tikzcd}
$$

De modo a não transformar a expressão orginal, achamos que o gene do catamorfismo deve criar um par com o valor original e o valor da derivada:

$$
\begin{array}{l}
ad_gen\ a = [g1\ a, g2]\ \mathbf{where} \\
\quad g1\ a = \underline{(a, 1)} \\
\quad g2 = [g3, g4]\ \mathbf{where} \\
\quad\quad g3\ a = (a, 0) \\
\quad\quad g4 = [g5, g6]\ \mathbf{where} \\
\quad\quad\quad g5\ (Sum, ((ex1, ex2), (ex3, ex4))) = (ex1 + ex3, ex2 + ex4) \\
\quad\quad\quad g5\ (Product, ((ex1, ex2), (ex3, ex4))) = (ex1 * ex3, ex1 * ex4 + ex2 * ex3) \\
\quad\quad\quad g6\ (Negate, (ex1, ex2)) = (negate\ ex1, negate\ ex2) \\
\quad\quad\quad g6\ (E, (ex1, ex2)) = (expd\ ex1, ex2 * (expd\ ex1))
\end{array}
$$

Problema 2

Definir

Seja C(0) = 1 e C(n+1) =

$$\frac{2*(2n+1)}{n+2}(f(x)+g(x)) = \frac{d}{dx}(f(x)) + \frac{d}{dx}(g(x))$$

Seja C(0) = 1 e

$$C(n+1) = \frac{2*(2n+1)}{n+2}$$

catF 0 = 1 catF (n+1) = (cat F n) * (cat S n)/(cat T n) catS 0 = 2 catS (n+1) = (cat S n) + 2 catT 0 = 2 catT (n+1) = (cat F n) + 2*n + 2

$$\begin{array}{l} loop :: Integral\ c \Rightarrow (c,c,c) \to (c,c,c) \\ loop\ (cF, cS, cT) = (cF * cS \div cT, cS + 4, cT + 1) \\ inic = (1,2,2) \\ prj\ (cF, cS, cT) = cF \end{array}$$

por forma a que

$$cat = prj \cdot \mathsf{for}\ loop\ inic$$

seja a função pretendida. **NB**: usar divisão inteira. Apresentar de seguida a justificação da solução encontrada.

Problema 3

$$\begin{array}{l} calcLine :: NPoint \to (NPoint \to OverTime\ NPoint) \\ calcLine = cataList\ h\ \mathbf{where} \\ \quad h = \bot \\ deCasteljau :: [NPoint] \to OverTime\ NPoint \\ deCasteljau = hyloAlgForm\ alg\ coalg\ \mathbf{where} \\ \quad coalg = \bot \\ \quad alg = \bot \\ hyloAlgForm = \bot \end{array}$$

Problema 4

$$avg = \pi_1 \cdot avg_aux$$

É-nos dito no enunciado que $avg_aux = (\!|[b,q]|\!) = \langle avg, length \rangle$, então, o raciocínio é partir de $[b,q]$ e tentar obter um split que permita ter um catamorfismo de um split passível de aplicação da regra Fokkinga. Assim:

$$\begin{array}{lll} & [b,q] & (9) \\ \equiv & \quad \{\ \text{Natural-id}\ \} & \\ & [id \cdot b, id \cdot q] & (10) \\ \equiv & \quad \{\ \text{Fusão-+}\ \} & \\ & id \cdot [b,q] & (11) \\ \equiv & \quad \{\ \text{Reflexão-x}\ \} & \\ & [\langle \pi_1, \pi_2 \rangle \cdot b, \langle \pi_1, \pi_2 \rangle \cdot q] & (12) \end{array}$$

$$\equiv \quad \{ \text{ Fusão-x, Natural id } \}$$

$$[\langle \pi_1 \cdot b, \pi_2 \cdot b \rangle, \langle \pi_1 \cdot q, \pi_2 \cdot q \rangle] \tag{13}$$

$$\equiv \quad \{ \text{ Lei da Troca } \}$$

$$\langle [\pi_1 \cdot b, \pi_1 \cdot q], [\pi_2 \cdot b, \pi_2 \cdot q] \rangle \tag{14}$$

$$\square \tag{15}$$

Tendo um catamofismo de um split, aplique-se Fokkinga:

$$\langle [\pi_1 \cdot b, \pi_1 \cdot q], [\pi_2 \cdot b, \pi_2 \cdot q] \rangle = \langle avg, length \rangle \tag{16}$$

$$\equiv \quad \{ \text{ Fokkinga } \}$$

$$\begin{cases} avg \cdot \mathbf{in} = [\pi_1 \cdot b, \pi_1 \cdot q] \cdot F \ \langle avg, length \rangle \\ length \cdot \mathbf{in} = [\pi_2 \cdot b, \pi_2 \cdot q] \cdot F \ \langle avg, length \rangle \end{cases} \tag{17}$$

$$\equiv \quad \{ \text{ Def in , Def F } \}$$

$$\begin{cases} avg \cdot [nil, cons] = [\pi_1 \cdot b, \pi_1 \cdot q] \cdot (id + id \times \langle avg, length \rangle) \\ length \cdot [nil, cons] = [\pi_2 \cdot b, \pi_2 \cdot q] \cdot (id + id \times \langle avg, length \rangle) \end{cases} \tag{18}$$

$$\equiv \quad \{ \text{ fusão-+ } \}$$

$$\begin{cases} [avg \cdot nil, avg \cdot cons] = [\pi_1 \cdot b, \pi_1 \cdot q] \cdot (id + id \times \langle avg, length \rangle) \\ [length \cdot nill, len \cdot cons] = [\pi_2 \cdot b, \pi_2 \cdot q] \cdot (id + id \times \langle avg, length \rangle) \end{cases} \tag{19}$$

$$\equiv \quad \{ \text{ Def-x } \}$$

$$\begin{cases} [avg \cdot nil, avg \cdot cons] = [\pi_1 \cdot b, \pi_1 \cdot q] \cdot (id + \langle id \cdot \pi_1, \langle avg, len \rangle \cdot \pi_2 \rangle) \\ [length \cdot nill, len \cdot cons] = [\pi_1 \cdot b, \pi_1 \cdot q]) \cdot (id + \langle id \cdot \pi_1, \langle avg, len \rangle \cdot \pi_2 \rangle) \end{cases} \tag{20}$$

$$\equiv \quad \{ \text{ Absorção-+ } \}$$

$$\begin{cases} [avg \cdot nil, avg \cdot cons] = [\pi_1 \cdot b \cdot id, \pi_1 \cdot q \cdot \langle avg, len \rangle \cdot \pi_2] \\ [length \cdot nill, len \cdot cons] = [\pi_2 \cdot b \cdot id, \pi_2 \cdot q \cdot \langle avg, len \rangle \cdot \pi_2] \end{cases} \tag{21}$$

$$\equiv \quad \{ \text{ Eq-+ , Natural-id } \}$$

$$\begin{cases} (avg \cdot nil) = (\pi_1 \cdot b) \\ (avg \cdot cons) = (\pi_1 \cdot q \cdot \langle avg, len \rangle \cdot \pi_2) \\ (length \cdot nill) = (\pi_2 \cdot b \cdot id) \\ (len \cdot cons) = (\pi_2 \cdot q \cdot \langle avg, len \rangle \cdot \pi_2) \end{cases} \tag{22}$$

$$\equiv \quad \{ \text{ Igualdade Extensional; Def-comp; Def split } \}$$

$$\begin{cases} avg \ (nil \ l) = \pi_1 \ (b \ l) \\ avg \ (cons \ (a, l)) = \pi_1 \ (q \ (a, (avg \ l, length \ l))) \\ length \ (nil \ l) = \pi_2 \ (b \ l) \\ length \ (cons \ (a, l)) = \pi_2 \ (q \ (a, (avg \ l, length \ l))) \end{cases} \tag{23}$$

$$\equiv \quad \{ \text{ Def nil, Def cons, avg e length [] = 0 } \}$$

$$\begin{cases} \pi_1 \ b = 0 \\ avg \ (a : l) = \pi_1 \ (q \ (a, (avg \ l, length \ l)) \\ \pi_2 \ b = 0 \\ length \ (a : l) = \pi_2 \ (q \ (a, (avg \ l, length \ l)) \end{cases} \tag{24}$$

$$\square \tag{25}$$

Como se compreende, com $\pi_1 \ b = 0$, qualquer que seja o b, o resultado é sempre 0, logo $b = \underline{b}$, ou seja, b é uma função constante, tendo-se como solução:

$$\begin{cases} \pi_1 \ \underline{b} = 0 \\ avg \ (a : l) = \pi_1 \ (q \ (a, (avg \ l, length \ l)) \\ \pi_2 \ \underline{b} = 0 \\ length \ (a : l) = \pi_2 \ (q \ (a, (avg \ l, length \ l)) \end{cases} \tag{26}$$

$$\square \tag{27}$$

Logo, tem-se a solução:

$$\begin{array}{l} avg_aux = cataList\ [\underline{b}, q]\ \mathbf{where} \\ \quad b = (0,0) \\ \quad q\ (c,(avg,len)) = ((avg * len + c)\ /\ (\mathsf{succ}\ \ len), \mathsf{succ}\ \ len) \end{array}$$

Confrontando com esta solução com o obtido anteriormente, percebe-se também que o $\underline{b}$ é o inicializador do ciclo *for* ($\underline{b} = (0,0)$) e o loop (b) é definido por $Def\ \ avg$ dada pelo enunciado e $Def\ \ length$ de uma lista.

Solução para árvores de tipo LTree:

Esta demonstração, em termos de leis é extremamente parecida com a anterior.

Através das expressões obtidas após a aplicação de Fokkinga, temos:

$$\begin{array}{ll} & \left\{ \begin{array}{l} avg \cdot \mathbf{in} = [\pi_1 \cdot \underline{b}, \pi_1 \cdot q] \cdot F\ \langle avg, length \rangle \\ len \cdot \mathbf{in} = [\pi_2 \cdot \underline{b}, \pi_2 \cdot q] \cdot F\ \langle avg, length \rangle \end{array} \right. \\ \equiv & \quad \{ \text{ Def in , Def F } \} \\ & \left\{ \begin{array}{l} avg \cdot [Leaf, Fork] = [\pi_1 \cdot \underline{b}, \pi_1 \cdot q] \cdot (id + (\langle avg, length \rangle \times \langle avg, length \rangle)) \\ len \cdot [Leaf, Fork] = [\pi_2 \cdot \underline{b}, \pi_2 \cdot q] \cdot (id + (\langle avg, length \rangle \times \langle avg, length \rangle)) \end{array} \right. \\ \equiv & \quad \{ \text{ Fusão-+; Def-x; Absorção-+; Eq-+; Natural-id; Igualdade Extensional; Def split; Def-comp } \} \\ & \left\{ \begin{array}{l} avg\ (Leaf\ \ l) = \pi_1 \cdot \underline{b}\ l \\ avg\ (Fork\ (esq, dir)) = \pi_1\ (t\ ((avg\ \ esq, length\ \ esq), (avg\ \ dir, length\ \ dir))) \\ length\ (Leaf\ \ l) = \pi_2 \cdot \underline{b}\ l \\ length\ (Fork\ (esq, dir)) = \pi_2\ (t\ ((avg\ \ esq, length\ \ esq), (avg\ \ dir, length\ \ dir))) \end{array} \right. \\ \equiv & \quad \{ \text{ Def avg ; length leaf l = (l,1) } \} \\ & \left\{ \begin{array}{l} l = \pi_1\ b \\ avg\ (Fork\ (esq, dir)) = \pi_1\ (t\ ((avg\ \ esq, length\ \ esq), (avg\ \ dir, length\ \ dir))) \\ 1 = \pi_2\ b \\ length\ (Fork\ (esq, dir)) = \pi_2\ (t\ ((avg\ \ esq, length\ \ esq), (avg\ \ dir, length\ \ dir))) \end{array} \right. \\ \square & \end{array}$$

Conclui-se, então $length\ \ Fork = length\ (SubRightTree) + length\ (SubLeftTree)$

$$\begin{array}{l} avgLTree = \pi_1 \cdot (\!|\, gene \,|\!)\ \mathbf{where} \\ \quad gene = [b, t] \\ \quad b\ \ a = (a, 1) \\ \quad t\ ((esq1, esq2), (dir1, dir2)) = ((esq1 * esq2 + dir1 * dir2)\ /\ (esq2 + dir2), esq2 + dir2) \end{array}$$

Problema 5

Inserir em baixo o código F# desenvolvido, entre `\begin{verbatim}` e `\end{verbatim}`: